

Experiment No.	1
Experiment Title	Exploratory Data Analysis – Spambase Dataset ¹
Student Name	Sanghamithra Namboodiri
Register Number	3122247001055
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	3rd August 2026

1 Aim and Objective

To explore and understand the functionalities of core Python libraries used in Machine Learning – NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib/Seaborn – and to apply exploratory data analysis (EDA) techniques (data cleaning, outlier removal, visualization, correlation analysis) on the Spambase dataset in order to identify the type of ML task it represents and the algorithms best suited to it in later experiments.

2 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository
Number of Samples	4601 (raw); 4210 after duplicate removal; 809 after IQR-based outlier removal
Number of Features	57 numeric predictor features (48 <code>word_freq_*</code> , 6 <code>char_freq_*</code> , 3 <code>capital_run_length_*</code>)
Number of Classes	2 (0 = non-spam, 1 = spam)
Missing Values	0 (before and after cleaning)
Train-Test Split	Not applicable – this experiment covers EDA only; no model is trained

The dataset contains 4601 emails described by 57 numeric predictor features (55 of type `float64` and 2 of type `int64`) plus a binary integer `class` label, occupying roughly 2.0 MB in memory. The statistical summary shows that most `word_freq_*` columns have a median of 0, confirming that they are heavily right-skewed and zero-inflated, while `class` has a mean of about 0.399, matching the roughly 76:24 non-spam/spam split observed after cleaning.

3 Preprocessing Steps

- **Missing value handling:** None required; all 58 columns show 0 missing values, both before and after cleaning.
- **Duplicate removal:** 391 duplicate rows were identified and dropped, reducing the dataset from 4601 to 4210 rows.
- **Outlier handling:** Outliers were flagged and removed on a per-column basis using the IQR method; columns with $IQR = 0$ (sparse, zero-inflated columns) were

skipped rather than filtered, since IQR-based bounds are undefined/degenerate for them. This reduced the dataset from 4210 to 809 rows – an 81% reduction.

- **Feature scaling:** Not performed in this experiment, since no model is trained here; scaling (`MinMaxScaler`) is applied in the subsequent modelling experiment.
- **Train-test split:** Not applicable to this experiment, as it covers EDA only.
- **Encoding:** Not applicable; the target label was already numeric (0/1), and all 57 predictors were already numeric.

Table 2: Dataset shape at each cleaning stage

Stage	Rows	Columns
Original	4601	58
After missing value removal	4601	58
After duplicate removal	4210	58
After IQR outlier removal	809	58

4 Implementation Details

Environment

Python Version	3.x
Pandas / NumPy	Standard current release
IDE	VS Code
Operating System	Linux
Hardware	Standard CPU

Pipeline

The EDA pipeline (`eda.py`) was implemented as follows:

1. Load `spambase_csv.csv` into a Pandas DataFrame.
2. Inspect structure with `.head()`, `.info()`, `.shape`, `.columns`, `.dtypes`, `.nunique()`.
3. Check missing values with `.isnull().sum()` before and after cleaning.
4. Identify and drop duplicate rows with `.duplicated()` / `.drop_duplicates()`.
5. Compute descriptive statistics with `.describe()`.
6. Detect and remove outliers per column using the IQR rule ($Q1 - 1.5 \times \text{IQR}$, $Q3 + 1.5 \times \text{IQR}$), skipping columns where $\text{IQR} = 0$.
7. Generate visualizations (correlation heatmap, feature distributions, class balance) using Matplotlib/Seaborn.

Console Output

```
~/ML-EXP1 >> python3 eda.py
=====
EXPLORATORY DATA ANALYSIS
=====

First 5 Rows
  word_freq_make  word_freq_address  ...  capital_run_length_total  class
0             0.00             0.64  ...                278         1
1             0.21             0.28  ...               1028         1
2             0.06             0.00  ...               2259         1
3             0.00             0.00  ...                191         1
4             0.00             0.00  ...                191         1

[5 rows x 58 columns]

Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 4601 entries, 0 to 4600
Data columns (total 58 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   word_freq_make                           4601 non-null   float64
1   word_freq_address                       4601 non-null   float64
2   word_freq_all                           4601 non-null   float64
3   word_freq_3d                            4601 non-null   float64
4   word_freq_our                           4601 non-null   float64
5   word_freq_over                          4601 non-null   float64
6   word_freq_remove                        4601 non-null   float64
7   word_freq_internet                      4601 non-null   float64
8   word_freq_order                         4601 non-null   float64
9   word_freq_mail                          4601 non-null   float64
10  word_freq_receive                       4601 non-null   float64
11  word_freq_will                          4601 non-null   float64
12  word_freq_people                        4601 non-null   float64
13  word_freq_report                        4601 non-null   float64
14  word_freq_addresses                     4601 non-null   float64
15  word_freq_free                          4601 non-null   float64
16  word_freq_business                      4601 non-null   float64
17  word_freq_email                         4601 non-null   float64
18  word_freq_you                           4601 non-null   float64
19  word_freq_credit                        4601 non-null   float64
20  word_freq_your                          4601 non-null   float64
21  word_freq_font                          4601 non-null   float64
22  word_freq_000                           4601 non-null   float64
23  word_freq_money                         4601 non-null   float64
24  word_freq_hp                            4601 non-null   float64
25  word_freq_hpl                           4601 non-null   float64
26  word_freq_george                        4601 non-null   float64
27  word_freq_650                           4601 non-null   float64
28  word_freq_lab                           4601 non-null   float64
29  word_freq_labs                           4601 non-null   float64
30  word_freq_telnet                        4601 non-null   float64
31  word_freq_857                           4601 non-null   float64
32  word_freq_data                           4601 non-null   float64
33  word_freq_415                           4601 non-null   float64
34  word_freq_85                            4601 non-null   float64
35  word_freq_technology                    4601 non-null   float64
36  word_freq_1999                           4601 non-null   float64
37  word_freq_parts                          4601 non-null   float64
38  word_freq_pm                            4601 non-null   float64
39  word_freq_direct                        4601 non-null   float64
40  word_freq_cs                            4601 non-null   float64
41  word_freq_meeting                       4601 non-null   float64
42  word_freq_original                      4601 non-null   float64
43  word_freq_project                       4601 non-null   float64
44  word_freq_re                            4601 non-null   float64
45  word_freq_edu                           4601 non-null   float64
46  word_freq_table                         4601 non-null   float64
47  word_freq_conference                    4601 non-null   float64
48  char_freq_%3B                           4601 non-null   float64
49  char_freq_%28                           4601 non-null   float64
```

```

50 char_freq_%5B          4601 non-null float64
51 char_freq_%21          4601 non-null float64
52 char_freq_%24          4601 non-null float64
53 char_freq_%23          4601 non-null float64
54 capital_run_length_average 4601 non-null float64
55 capital_run_length_longest 4601 non-null int64
56 capital_run_length_total  4601 non-null int64
57 class                   4601 non-null int64
dtypes: float64(55), int64(3)
memory usage: 2.0 MB

Dataset Shape: (4601, 58)

Missing Values Before Cleaning
[all 58 columns show 0 missing values]

Missing Values After Cleaning
[all 58 columns show 0 missing values]

Duplicate Rows: 391
Shape After Removing Duplicates: (4210, 58)

Numerical Statistics
      word_freq_make word_freq_address ... capital_run_length_total class
count      4210.000000      4210.000000 ...      4210.000000 4210.000000
mean         0.104366         0.112656 ...         291.181948  0.398812
std          0.300005         0.454260 ...         618.654838  0.489712
min          0.000000         0.000000 ...          1.000000  0.000000
25%          0.000000         0.000000 ...          40.000000  0.000000
50%          0.000000         0.000000 ...         101.500000  0.000000
75%          0.000000         0.000000 ...         273.750000  1.000000
max          4.540000        14.280000 ...        15841.000000  1.000000

[8 rows x 58 columns]

Removing Outliers (IQR Method)
word_freq_make: skipped (IQR is 0, likely a sparse/constant column)
word_freq_address: skipped (IQR is 0, likely a sparse/constant column)
word_freq_all: 293 outliers flagged
word_freq_3d: skipped (IQR is 0, likely a sparse/constant column)
word_freq_our: 449 outliers flagged
word_freq_over: skipped (IQR is 0, likely a sparse/constant column)
word_freq_remove: skipped (IQR is 0, likely a sparse/constant column)
word_freq_internet: skipped (IQR is 0, likely a sparse/constant column)
word_freq_order: skipped (IQR is 0, likely a sparse/constant column)
word_freq_mail: 748 outliers flagged
word_freq_receive: skipped (IQR is 0, likely a sparse/constant column)
word_freq_will: 250 outliers flagged
word_freq_people: skipped (IQR is 0, likely a sparse/constant column)
word_freq_report: skipped (IQR is 0, likely a sparse/constant column)
word_freq_addresses: skipped (IQR is 0, likely a sparse/constant column)
word_freq_free: 830 outliers flagged
word_freq_business: skipped (IQR is 0, likely a sparse/constant column)
word_freq_email: skipped (IQR is 0, likely a sparse/constant column)
word_freq_you: 58 outliers flagged
word_freq_credit: skipped (IQR is 0, likely a sparse/constant column)
word_freq_your: 193 outliers flagged
word_freq_font: skipped (IQR is 0, likely a sparse/constant column)
word_freq_000: skipped (IQR is 0, likely a sparse/constant column)
word_freq_money: skipped (IQR is 0, likely a sparse/constant column)
word_freq_hp: 1012 outliers flagged
word_freq_hpl: skipped (IQR is 0, likely a sparse/constant column)
word_freq_george: skipped (IQR is 0, likely a sparse/constant column)
word_freq_650: skipped (IQR is 0, likely a sparse/constant column)
word_freq_lab: skipped (IQR is 0, likely a sparse/constant column)
word_freq_labs: skipped (IQR is 0, likely a sparse/constant column)
word_freq_telnet: skipped (IQR is 0, likely a sparse/constant column)
word_freq_857: skipped (IQR is 0, likely a sparse/constant column)
word_freq_data: skipped (IQR is 0, likely a sparse/constant column)
word_freq_415: skipped (IQR is 0, likely a sparse/constant column)
word_freq_85: skipped (IQR is 0, likely a sparse/constant column)
word_freq_technology: skipped (IQR is 0, likely a sparse/constant column)
word_freq_1999: skipped (IQR is 0, likely a sparse/constant column)

```

```

word_freq_parts: skipped (IQR is 0, likely a sparse/constant column)
word_freq_pm: skipped (IQR is 0, likely a sparse/constant column)
word_freq_direct: skipped (IQR is 0, likely a sparse/constant column)
word_freq_cs: skipped (IQR is 0, likely a sparse/constant column)
word_freq_meeting: skipped (IQR is 0, likely a sparse/constant column)
word_freq_original: skipped (IQR is 0, likely a sparse/constant column)
word_freq_project: skipped (IQR is 0, likely a sparse/constant column)
word_freq_re: 805 outliers flagged
word_freq_edu: skipped (IQR is 0, likely a sparse/constant column)
word_freq_table: skipped (IQR is 0, likely a sparse/constant column)
word_freq_conference: skipped (IQR is 0, likely a sparse/constant column)
char_freq_%3B: skipped (IQR is 0, likely a sparse/constant column)
char_freq_%28: 255 outliers flagged
char_freq_%5B: skipped (IQR is 0, likely a sparse/constant column)
char_freq_%21: 373 outliers flagged
char_freq_%24: 730 outliers flagged
char_freq_%23: skipped (IQR is 0, likely a sparse/constant column)
capital_run_length_average: 329 outliers flagged
capital_run_length_longest: 418 outliers flagged
capital_run_length_total: 497 outliers flagged

Shape Before Outlier Removal: (4210, 58)
Shape After Outlier Removal: (809, 58)

```

5 Visualizations

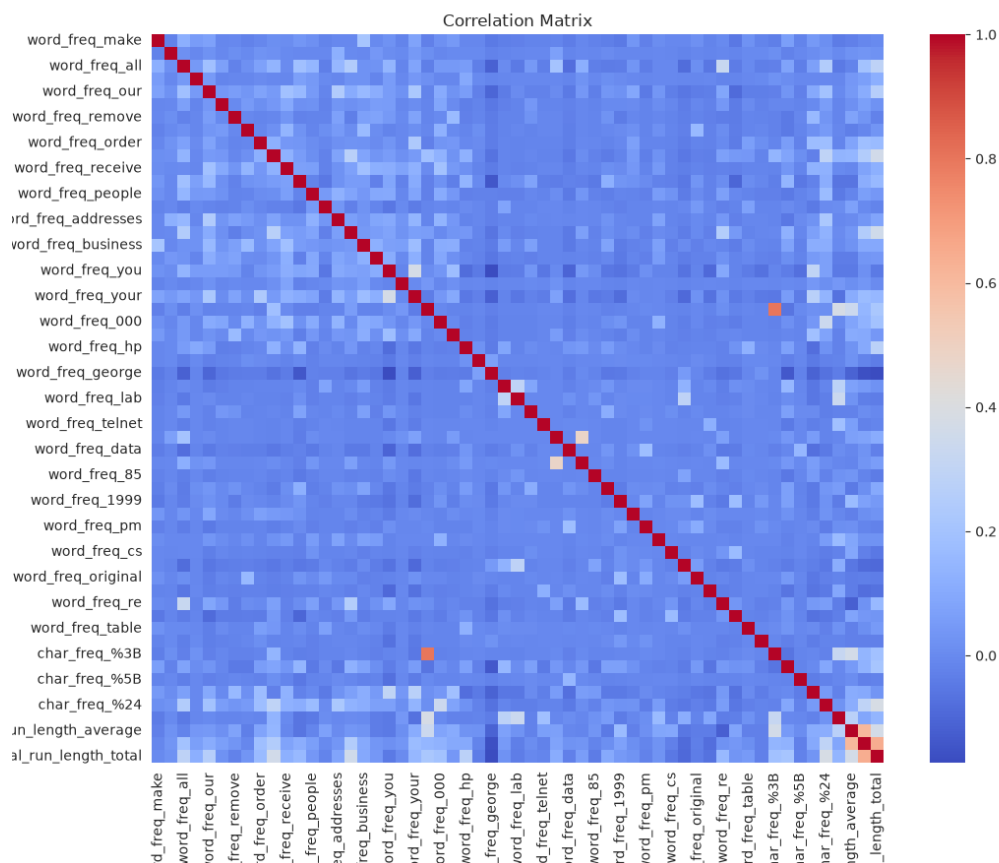


Figure 1: Correlation matrix of numeric features in the Spambase dataset.

Inference: Most feature pairs have near-zero correlation, indicating that multicollinearity is not a major concern for this dataset and that most of the 57 features can be retained

without redundancy concerns. A few related `word_freq_*` and `char_freq_*` pairs show mild positive correlation, consistent with spam emails reusing similar vocabulary and symbol patterns together. The largely diagonal-dominant structure suggests each feature carries relatively independent information, which is useful context for choosing classifiers later. This also supports treating the features as roughly independent predictors in subsequent modelling experiments.



Figure 2: Distribution of numeric `word_freq_*` and `char_freq_*` features.

Inference: Nearly all `word_freq_*` columns are heavily zero-inflated and right-skewed, since most individual words do not appear in most emails and values are therefore 0 for the majority of rows. This directly explains why 34 of the 57 columns were skipped during IQR-based outlier removal, as their first and third quartiles are both 0 ($IQR = 0$). The skewed, sparse shape of these distributions suggests count/frequency-oriented likelihood models will fit this data better than normality-based ones. It also motivates scaling all features to a common range before applying any distance-based algorithm.

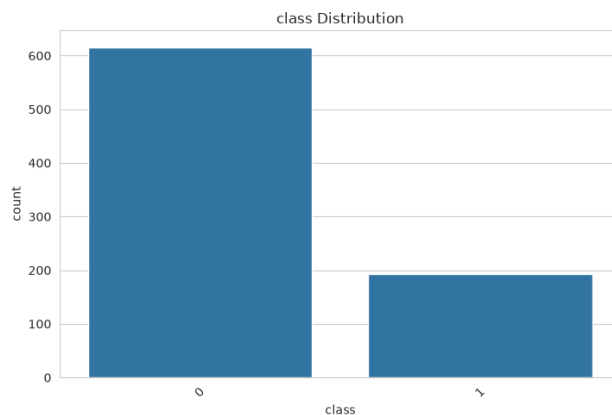


Figure 3: Class distribution after cleaning and outlier removal (0 = non-spam, 1 = spam).

Inference: After cleaning and outlier removal, the class distribution shows a moderate imbalance of roughly 76:24 (616 non-spam vs. 193 spam emails). This means accuracy alone would be a misleading evaluation metric for a downstream spam classifier, and precision, recall, F1-score, or ROC-AUC should be preferred. The imbalance also suggests that class-weighting or resampling strategies may be worth exploring in later modelling experiments. It further indicates that stratified splitting should be used when partitioning the data for training and testing.

6 Performance Tables

No classifier is trained in this experiment, so no accuracy/precision/ recall-style performance table applies. The only quantitative “performance” of the pipeline itself is the effect of each cleaning stage on dataset size, summarized below (repeated from Table 2 for reference).

Table 3: Effect of each preprocessing stage on dataset size

Stage	Rows	Rows Removed	% Removed (cumulative)
Original	4601	—	0.0%
After duplicate removal	4210	391	8.5%
After IQR outlier removal	809	3401	82.4%

7 Overfitting and Underfitting Analysis

Not applicable to this experiment. Overfitting and underfitting are properties of a trained model’s behaviour on training vs. unseen data, and no model is fit here – this experiment is limited to data inspection and cleaning. This analysis is carried out in Experiment 2, where Gaussian NB, Multinomial NB, Bernoulli NB, and KNN are actually trained on the data prepared here. What this experiment does establish, which is directly relevant to overfitting risk downstream, is that the IQR-based outlier removal step is aggressive (removing 82% of rows), which shrinks the effective training set to 809 rows and increases the risk of high-variance (overfit) models being trained on too little data in the next experiment.

8 Bias–Variance Analysis

Not applicable to this experiment for the same reason: bias and variance are properties of a fitted estimator’s predictions, which requires a trained model. No model is trained in this EDA-only experiment. However, the findings here inform the bias–variance trade-off expected in Experiment 2: the heavy right-skew and zero-inflation of most `word_freq_*` features (Figure 2) suggests that models assuming normality (e.g. Gaussian NB) will have high *bias* on this data, since their distributional assumption is a poor match; while the small post-cleaning sample size (809 rows) increases the risk of high *variance* for flexible, low-bias models such as KNN with a small k .

9 Observations and Conclusion

Observations. The correlation matrix shows that most feature pairs have near-zero correlation, so multicollinearity is not a major concern and most features can be retained as-is for downstream modelling. The numeric feature distributions reveal that nearly all `word_freq_*` columns are heavily zero-inflated and right-skewed, which is the direct cause of 34 of the 57 columns being skipped during IQR-based outlier removal (their IQR was 0). The class distribution shows a moderate 76:24 imbalance after cleaning, meaning accuracy alone is a misleading metric for any classifier subsequently trained on this data.

A key limitation observed is that IQR-based outlier removal reduced the dataset from 4210 rows (after deduplication) to just 809 rows, an 81% reduction, driven by the compounding effect of applying IQR filtering across 34 skewed columns simultaneously. This aggressive reduction risks discarding genuinely informative (if extreme) examples, and a milder filtering approach – such as a wider IQR multiplier, or skipping outlier removal on skewed, zero-inflated count-like features entirely – would likely preserve more data for downstream classification.

Conclusion. This experiment demonstrated the practical use of NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib/Seaborn to explore, clean, and visualize the Spambase dataset. Key findings were: (1) no missing values were present in the raw data; (2) 391 duplicate rows were removed; (3) IQR-based outlier removal is highly aggressive on this dataset, reducing it by 81% due to its skewed, zero-inflated feature distributions; (4) most features are weakly correlated, so multicollinearity is not a major concern; and (5) the cleaned dataset exhibits a moderate 76:24 class imbalance that should inform metric choice and possibly resampling strategy in subsequent modelling experiments. These findings directly motivate the choice of count/frequency-oriented classifiers (e.g. Multinomial and Bernoulli Naive Bayes) and imbalance-aware evaluation metrics used in the following experiment.